

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ace7351e-c4e\agent-aa431c1f112704946.jsonl`
- SHA-256: `0804bb4053059afb94dcdbc23cd5b8abaa55a88ffcda794a52a2aa4fa0d4646b`
- Source modified: `2026-07-06T22:20:26+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_ace7351e-c4e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T22:19:43.196Z)

Reviewing GitHub PR #52 "feat(policy): add F3 caption safety stage" (branch feat/f3-caption-safety -> main). ADR 0012 F3.
+299/-41, 4 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr52 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f3-caption-safety:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F3 DOES:

- policy.py: CaptionSafetyConfig + parse_caption_safety_config (from 'security_checks' key: unsafe_caption_terms[], unsafe_caption_patterns[]);
CaptionSafetyStage (Tier 3): fail-OPEN on invalid config (pass_ + warning + invalid_caption_safety_config=True); disabled if empty;
missing caption -> pass (caption_present=False); term match = casefold substring; pattern match = re.search(pattern, caption) (case-SENSITIVE
unless user adds (?i)); regexes validated via re.compile at parse time (invalid regex -> ValueError -> fail-open).
- telethon_client.py: _evaluate_topic_preferences RENAMED to _evaluate_policy_stages, now runs [TopicPreferenceStage(), CaptionSafetyStage()]
through evaluate_pipeline() and returns the list; live+backfill wiring: 'if policy_results: reasons.extend(...); matched = policy_results[-1].passed'.
- Reads security_checks from the RAW policy config (preserved by F1's save merge).
on_new_message/backfill_user are '# pragma: no cover'.

CONFIRMED by the main reviewer: gate green (523 passed @ 95.77%); ReDoS is REAL ? the valid pattern '(a+)+\$' on a 40-char caption hangs
re.search past a 6s timeout (rc=124), blocking the synchronous event loop. Feature is dormant today (no command sets security_checks).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.
Severity: high (real bug/regression/exploitable-now), medium (real but latent / host-wide-DoS-once-configurable / design), low (hygiene).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if handled / can't occur /
pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity (recall: dormant-but-real host-DoS = medium, hygiene = low).
FINDING (security): Term matching is case-insensitive but pattern matching is case-sensitive ?
silent evasion gap in a safety stage
severity medium | src/tg_video_filter/policy.py:293

summary: unsafe_caption_terms are matched via casefold substring (case-insensitive), while unsafe_caption_patterns are matched via re.search on the un-casefolded caption (case-sensitive unless the operator remembers to prepend (?i)). An operator configuring a pattern to block content gets silently case-sensitive matching, which an attacker trivially bypasses by changing case.

scenario: Operator sets unsafe_caption_patterns to ['badword'] intending to block that word. A caption 'This is a BADWORD here' passes the safety stage (re.search('badword', 'This is a BADWORD here') is None), whereas the equivalent term ['badword'] correctly blocks the same caption. Because this is a safety/blocking stage, the case-sensitive pattern default is a fail-open evasion: an attacker just varies capitalization.

evidence: Repro (scratchpad/case.py): TERM 'badword' vs caption 'BADWORD' -> passed=False (blocked); PAT 'badword' vs caption 'BADWORD' -> passed=True (not blocked). Code: terms use haystack=caption.casefold() then `term.casefold() in haystack` (policy.py:290-292); patterns use `re.search(pattern, caption)` against the original-case caption (policy.py:293-294). Latent because the feature is dormant (no command sets security_checks) and behavior is operator-configurable, hence medium.

rec: Make pattern matching case-insensitive by default for consistency with term matching (compile with re.IGNORECASE), or at minimum document the asymmetry prominently and surface it in the config UI/command when one is added. If preserving raw regex semantics is intentional, keep term and pattern casing consistent (either both case-sensitive or both insensitive) so the safety contract is predictable.

2. assistant (2026-07-06T22:19:45.575Z)

I'll verify this finding adversarially. Let me read the cited code and run a repro.

3. user (2026-07-06T22:19:46.563Z)

```
255         except (json.JSONDecodeError, ValueError) as exc:
256             log.warning(
257                 "caption safety disabled due to invalid config policy_id=%s",
258                 ctx.policy.id,
259                 exc_info=True,
260             )
261             return StageResult.pass_(
262                 self.stage_name,
263                 f"invalid caption safety config ignored: {exc}",
264                 invalid_caption_safety_config=True,
265             )
266         if not config.enabled:
267             return StageResult.pass_(self.stage_name, "caption safety disabled")
268
269         caption = (ctx.content_item.text or "").strip()
270         if not caption:
271             return StageResult.pass_(
272                 self.stage_name,
273                 "no caption to scan",
274                 caption_present=False,
275             )
276
277         haystack = caption.casefold()
278         for term in config.unsafe_terms:
279             if term.casefold() in haystack:
280                 return StageResult.fail(
281                     self.stage_name,
282                     f"unsafe caption term matched: {term}",
283                     caption_present=True,
284                     match_type="term",
285                     matched_caption_term=term,
286                 )
287
288         for pattern in config.unsafe_patterns:
289             if re.search(pattern, caption):
290                 return StageResult.fail(
291                     self.stage_name,
```

```

292             "unsafe caption pattern matched",
293             caption_present=True,
294             match_type="pattern",
295             matched_caption_pattern=pattern,
296         )
297
298     return StageResult.pass_(
299         self.stage_name,
300         "caption passed safety checks",
301         caption_present=True,
302         checked_terms=len(config.unsafe_terms),
303         checked_patterns=len(config.unsafe_patterns),
304     )
305
306
307     async def evaluate_pipeline(
308         ctx: PolicyContext,
309         stages: list[PolicyStage],
310     ) -> tuple[bool, list[StageResult]]:
311         """Evaluate stages in order and short-circuit on the first failure."""
312         results: list[StageResult] = []
313         for stage in stages:
314             result = await stage.evaluate(ctx)
315             results.append(result)
316             if not result.passed:
317                 return False, results
318         return True, results
319
320
321     __all__ = [
322         "CaptionSafetyConfig",
323         "CaptionSafetyStage",
324         "PolicyContext",

```

4. assistant (2026-07-06T22:19:49.560Z)

Confirmed the code at policy.py:277-296. Terms use `caption.casefold()` haystack with `term.casefold()`, patterns use `re.search(pattern, caption)` on the un-casefolded caption. Let me run the repro.

5. user (2026-07-06T22:19:50.736Z)

```

200
201     haystack = _topic_haystack(ctx)
202     matched_excludes = _matched_topics(prefs.exclude, haystack)
203     if matched_excludes:
204         return StageResult.fail(
205             self.stage_name,
206             f"excluded topic matched: {matched_excludes[0]}",
207             matched_exclude_topics=list(matched_excludes),
208         )
209
210     if not prefs.include:
211         return StageResult.pass_(self.stage_name, "no include topics configured")
212
213     matched_includes = _matched_topics(prefs.include, haystack)
214     if prefs.mode == "any":
215         if matched_includes:
216             return StageResult.pass_(
217                 self.stage_name,
218                 f"matched include topic: {matched_includes[0]}",
219                 matched_include_topics=list(matched_includes),
220             )
221         return StageResult.fail(self.stage_name, "no include topics matched")
222

```

```

223         missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
224         if missing:
225             return StageResult.fail(
226                 self.stage_name,
227                 f"missing required topic: {missing[0]}",
228                 matched_include_topics=list(matched_includes),
229                 missing_include_topics=list(missing),
230             )
231         return StageResult.pass_(
232             self.stage_name,
233             "matched all include topics",
234             matched_include_topics=list(matched_includes),
235         )
236
237
238     def _topic_haystack(ctx: PolicyContext) -> str:
239         parts = [ctx.source.title or "", ctx.content_item.text or ""]
240         return "\n".join(parts).casefold()
241
242
243     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
244         return tuple(topic for topic in topics if topic.casefold() in haystack)
245
246
247     class CaptionSafetyStage:
248         """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
249
250         stage_name = "caption-safety"
251
252         async def evaluate(self, ctx: PolicyContext) -> StageResult:
253             try:
254                 config = load_caption_safety_config(ctx.policy)

```

6. assistant (2026-07-06T22:19:52.592Z)

Now let me look at how the config is parsed to write an accurate repro, then run it.

7. user (2026-07-06T22:19:54.683Z)

```

82: class CaptionSafetyConfig:
85:     unsafe_terms: tuple[str, ...] = ()
86:     unsafe_patterns: tuple[str, ...] = ()
89:     return bool(self.unsafe_terms or self.unsafe_patterns)
114: def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
118:     ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
126:     unsafe_terms = _string_tuple(
127:         raw_security.get("unsafe_caption_terms"),
128:         field_name="security_checks.unsafe_caption_terms",
130:     unsafe_patterns = _string_tuple(
131:         raw_security.get("unsafe_caption_patterns"),
132:         field_name="security_checks.unsafe_caption_patterns",
134:     for pattern in unsafe_patterns:
139:         "security_checks.unsafe_caption_patterns contains invalid regex "
143:         unsafe_terms=unsafe_terms,
144:         unsafe_patterns=unsafe_patterns,
156: def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
254:     config = load_caption_safety_config(ctx.policy)
278:     for term in config.unsafe_terms:
288:     for pattern in config.unsafe_patterns:
302:         checked_terms=len(config.unsafe_terms),
303:         checked_patterns=len(config.unsafe_patterns),
330:     "load_caption_safety_config",

```

8. assistant (2026-07-06T22:19:58.005Z)

The code is clear. I can verify the behavior directly with a minimal repro that mirrors the exact matching logic (term via casefold substring, pattern via `re.search` on original caption). Let me confirm empirically.

9. user (2026-07-06T22:20:03.955Z)

```
TERM 'badword' vs 'BADWORD' -> blocked = True
PAT  'badword' vs 'BADWORD' -> blocked = False
```

10. assistant (2026-07-06T22:20:07.537Z)

The repro confirms the asymmetry. Now let me adversarially test whether the finding truly holds ? could this be a false claim? Let me verify against the actual stage via the real code path to rule out that some upstream normalization casefolds the caption before it reaches `re.search`.

11. user (2026-07-06T22:20:08.144Z)

```
114     def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
115         """Parse caption safety settings from ``security_checks``.
116
117         Expected shape:
118         ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
119         """
120         raw_security = config.get("security_checks")
121         if raw_security is None:
122             return CaptionSafetyConfig()
123         if not isinstance(raw_security, dict):
124             raise ValueError("security_checks must be a JSON object")
125
126         unsafe_terms = _string_tuple(
127             raw_security.get("unsafe_caption_terms"),
128             field_name="security_checks.unsafe_caption_terms",
129         )
130         unsafe_patterns = _string_tuple(
131             raw_security.get("unsafe_caption_patterns"),
132             field_name="security_checks.unsafe_caption_patterns",
133         )
134         for pattern in unsafe_patterns:
135             try:
136                 re.compile(pattern)
137             except re.error as exc:
138                 raise ValueError(
139                     "security_checks.unsafe_caption_patterns contains invalid regex "
140                     f"{{pattern!r}}: {{exc}}"
141                 ) from exc
142         return CaptionSafetyConfig(
143             unsafe_terms=unsafe_terms,
144             unsafe_patterns=unsafe_patterns,
145         )
146
147
148     def load_topic_preferences(policy: Policy) -> TopicPreferences:
149         """Load topic preferences from ``policy.config_json``."""
150         decoded = json.loads(policy.config_json)
151         if not isinstance(decoded, dict):
152             raise ValueError("policy config_json must decode to a JSON object")
153         return parse_topic_preferences(decoded)
154
155
156     def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
157         """Load caption safety settings from ``policy.config_json``."""
158         decoded = json.loads(policy.config_json)
```

12. assistant (2026-07-06T22:20:19.008Z)

Config parsing does not casefold patterns (no ``lower()'`/``casefold()'` at parse or store time). ``re.compile(pattern)'` at line 136 is called without ``re.IGNORECASE'`, and the stored pattern string is passed verbatim to ``re.search(pattern, caption)'` at line 289 against the un-casefolded stripped caption. There is no upstream normalization that would casefold the caption before pattern matching.

The finding is confirmed. Assessing severity: this is a real asymmetry in a safety/blocking stage where an operator's mental model (terms are case-insensitive) does not transfer to patterns, creating a trivially-bypassable evasion gap. But it is operator-configurable and dormant (no command sets ``security_checks'` today), and the code does exhibit exactly the documented behavior ? an operator who writes a case-sensitive regex without ``(?i)'` gets case-sensitive matching. This matches the "dormant-but-real / design inconsistency" band = medium.

One nuance worth noting for honesty: regex case-sensitivity is arguably the **conventional** default for regexes (Python's ``re.search'` is case-sensitive by design), so a security-conscious operator might expect it. But paired with case-insensitive term matching in the same stage, the inconsistency is a genuine latent evasion trap. The defect (silent, undocumented asymmetry in a safety contract) genuinely ships in this PR's diff.

13. user (2026-07-06T22:20:26.810Z)

Structured output provided successfully